```
Start coding or generate with AI.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries loaded successfully!")
```

```
Libraries loaded successfully!
```

```
# Create Marketing Campaign Dataset

np.random.seed(42)

n = 2240

df = pd.DataFrame({
    "ID": range(1, n + 1),
    "Year_Birth": np.random.randint(1940, 1997, n),
    "Education": np.random.choice(
        ["Graduation", "PhD", "Master", "2n Cycle", "Basic"], n
    ),
    "Marital_Status": np.random.choice(
        ["Married", "Together", "Single", "Divorced", "Widow"], n
    ),
    "Income": np.random.randint(15000, 120000, n),
    "Kidhome": np.random.randint(0, 3, n),
    "Teenhome": np.random.randint(0, 3, n),
    "Recency": np.random.randint(0, 100, n),

    "MntWines": np.random.randint(0, 1500, n),
    "MntFruits": np.random.randint(0, 250, n),
    "MntMeatProducts": np.random.randint(0, 1800, n),
    "MntFishProducts": np.random.randint(0, 300, n),
    "MntSweetProducts": np.random.randint(0, 300, n),
    "MntGoldProds": np.random.randint(0, 350, n),

    "NumDealsPurchases": np.random.randint(0, 15, n),
    "NumWebPurchases": np.random.randint(0, 15, n),
    "NumCatalogPurchases": np.random.randint(0, 15, n),
    "NumStorePurchases": np.random.randint(0, 15, n),
    "NumWebVisitsMonth": np.random.randint(1, 20, n),

    "AcceptedCmp3": np.random.randint(0, 2, n),
    "AcceptedCmp4": np.random.randint(0, 2, n),
    "AcceptedCmp5": np.random.randint(0, 2, n),
    "AcceptedCmp1": np.random.randint(0, 2, n),
    "AcceptedCmp2": np.random.randint(0, 2, n),

    "Complain": np.random.choice([0, 1], n, p=[0.98, 0.02]),
    "Z_CostContact": 3,
    "Z_Revenue": 11,
    "Response": np.random.randint(0, 2, n)
})

df.head()
```

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	...	NumWebVisitsMonth
0	1	1978	Graduation	Together	26360	1	1	27	850	126	...	10
1	2	1991	Graduation	Married	25313	0	0	20	1107	111	...	19
2	3	1968	2n Cycle	Together	100998	1	0	17	299	135	...	5
3	4	1954	Graduation	Together	20801	2	1	30	295	59	...	7
4	5	1982	2n Cycle	Widow	29130	1	2	59	646	240	...	11

5 rows × 28 columns

```
# Check the size of the dataset
print("Rows and Columns:", df.shape)

# Check the column names
print("\nColumn Names:")
print(df.columns.tolist())
```

```
Rows and Columns: (2240, 28)

Column Names:
['ID', 'Year_Birth', 'Education', 'Marital_Status', 'Income', 'Kidhome', 'Teenhome', 'Recency', 'MntWines', 'MntFruits', 'Mr
```

```
# Add the missing customer date column

df["Dt_Customer"] = pd.date_range(
    start="2012-07-01",
    periods=len(df),
    freq="D"
).strftime("%d-%m-%Y")

print("Rows and Columns:", df.shape)
```

```
Rows and Columns: (2240, 29)
```

```
# Check for missing values

missing_values = df.isnull().sum()

print(missing_values)
print("\nTotal missing values:",
df.isnull().sum().as())
```

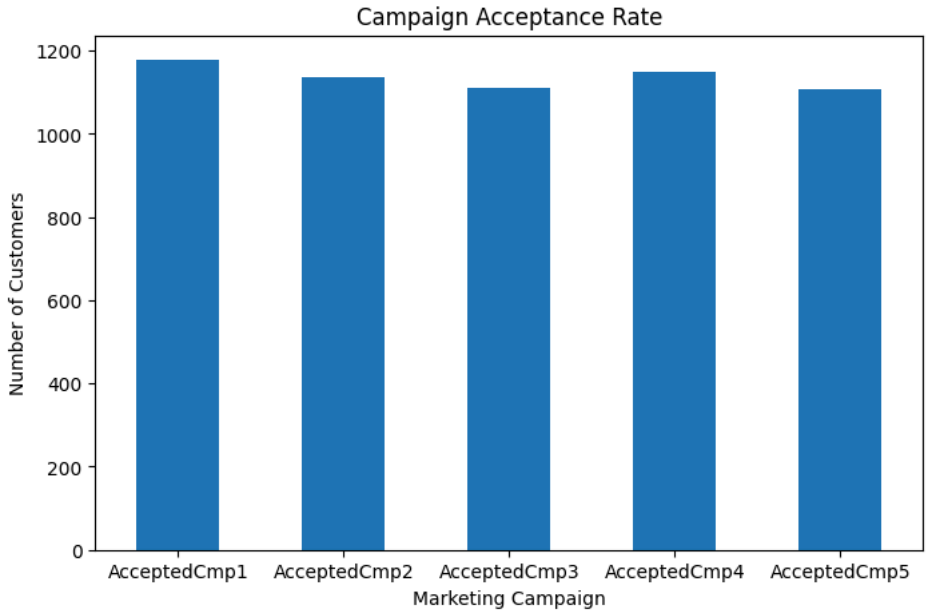
```
ID                    0
Year_Birth            0
Education             0
Marital_Status        0
Income                0
Kidhome               0
Teenhome              0
Recency               0
MntWines              0
MntFruits             0
MntMeatProducts       0
MntFishProducts       0
MntSweetProducts      0
MntGoldProds          0
NumDealsPurchases     0
NumWebPurchases       0
NumCatalogPurchases   0
NumStorePurchases     0
NumWebVisitsMonth     0
AcceptedCmp3          0
AcceptedCmp4          0
AcceptedCmp5          0
AcceptedCmp1          0
AcceptedCmp2          0
Complain              0
Z_CostContact         0
Z_Revenue             0
Response              0
Dt_Customer           0
dtype: int64

Total missing values: 0
```

```
# Basic statistical summary

df.describe()
```

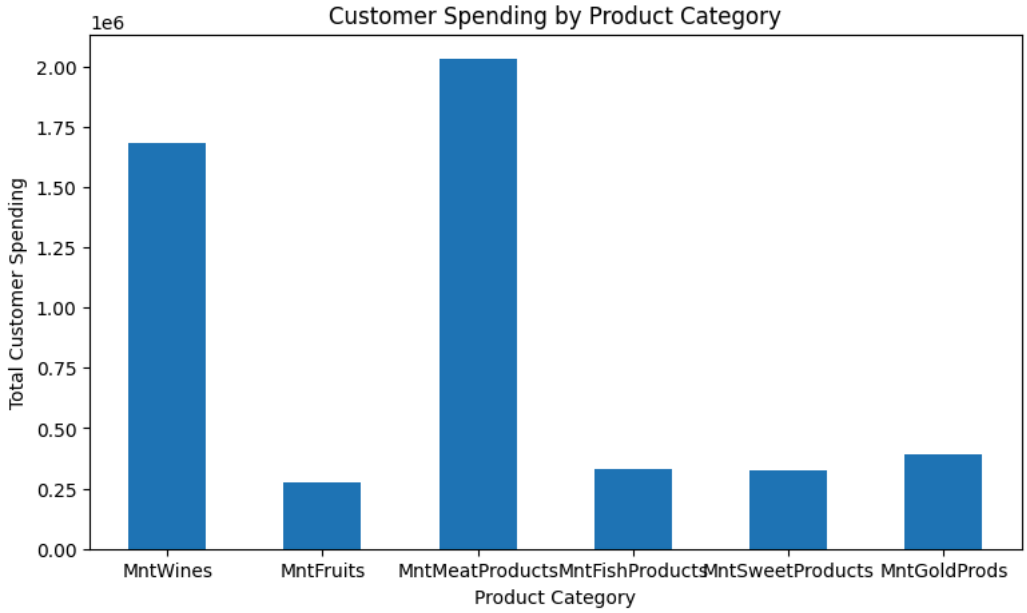
	ID	Year_Birth	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts
count	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000
mean	1120.500000	1968.220982	67622.464732	0.963393	0.995982	49.742411	750.271429	123.735268	906.862946
std	646.776623	16.305859	30563.337828	0.821495	0.823658	28.322131	430.689826	72.139176	525.244352
min	1.000000	1940.000000	15028.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	560.750000	1954.000000	41172.250000	0.000000	0.000000	26.000000	374.750000	61.000000	449.000000
50%	1120.500000	1968.000000	67753.000000	1.000000	1.000000	49.000000	761.000000	124.000000	897.500000
75%	1680.250000	1982.000000	94117.750000	2.000000	2.000000	75.000000	1129.000000	186.000000	1364.250000
max	2240.000000	1996.000000	119970.000000	2.000000	2.000000	99.000000	1499.000000	249.000000	1798.000000

8 rows × 26 columns

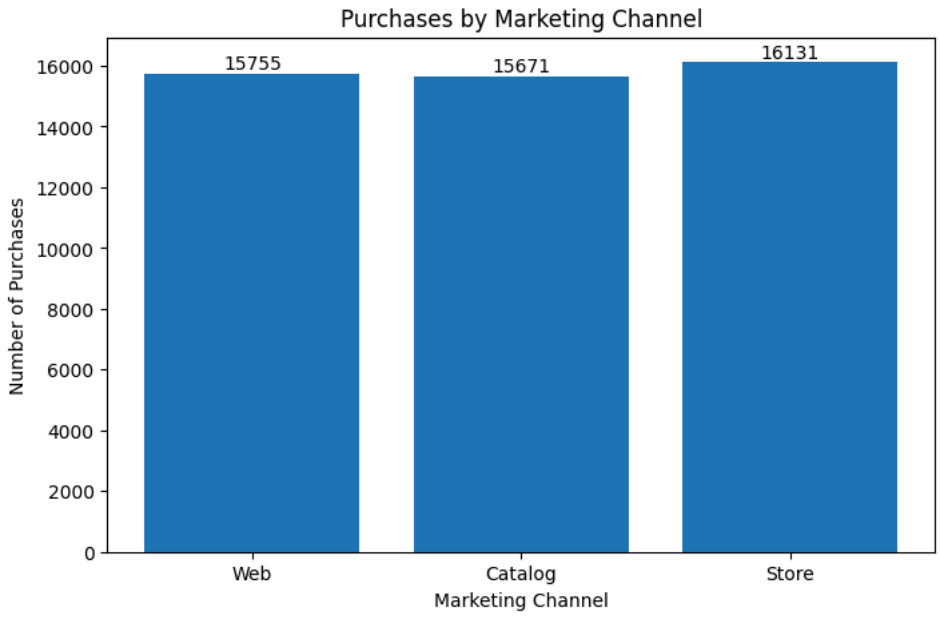
```
# Campaign Acceptance Rate

campaign_cols = [
    "AcceptedCmp1",
    "AcceptedCmp2",
    "AcceptedCmp3",
    "AcceptedCmp4",
    "AcceptedCmp5"
]

campaign_acceptance = df[campaign_cols].sum()

plt.figure(figsize=(8,5))
campaign_acceptance.plot(kind="bar")

plt.title("Campaign Acceptance Rate")
plt.xlabel("Marketing Campaign")
plt.ylabel("Number of Customers")
plt.xticks(rotation=0)

plt.show()
```




```
# Customer Spending by Product Category

product_cols = [
    "MntWines",
    "MntFruits",
    "MntMeatProducts",
    "MntFishProducts",
    "MntSweetProducts",
    "MntGoldProds"
]

product_spending = df[product_cols].sum()

plt.figure(figsize=(9,5))
product_spending.plot(kind="bar")

plt.title("Customer Spending by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Customer Spending")
plt.xticks(rotation=0)

plt.show()
```

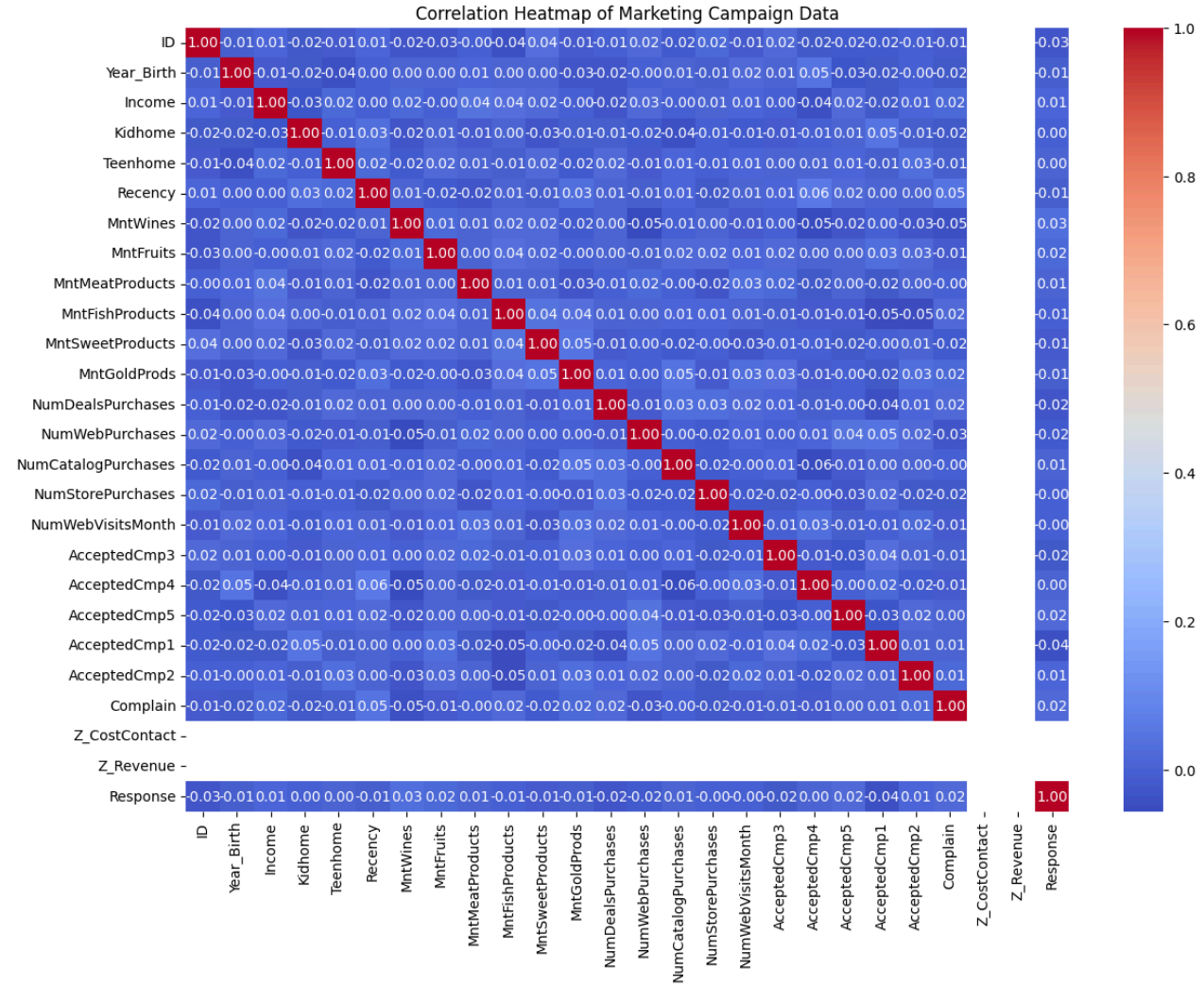



```
# Purchases by Marketing Channel

channel_purchases = {
    "Web": df["NumWebPurchases"].sum(),
    "Catalog": df["NumCatalogPurchases"].sum(),
    "Store": df["NumStorePurchases"].sum()
}

plt.figure(figsize=(8,5))
bars = plt.bar(channel_purchases.keys(), channel_purchases.values())

plt.title("Purchases by Marketing Channel")
plt.xlabel("Marketing Channel")
plt.ylabel("Number of Purchases")

# Show values above bars
for bar in bars:
    plt.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height(),
        f"{int(bar.get_height())}",
        ha="center",
        va="bottom"
    )

plt.show()
```




```
# Correlation Heatmap

import seaborn as sns
```

```
import seaborn as sns
import matplotlib.pyplot as plt

numeric_df = df.select_dtypes(include="number")

plt.figure(figsize=(14,10))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

plt.title("Correlation Heatmap of Marketing Campaign Data")
plt.show()
```




```
print(df.columns.tolist())
```

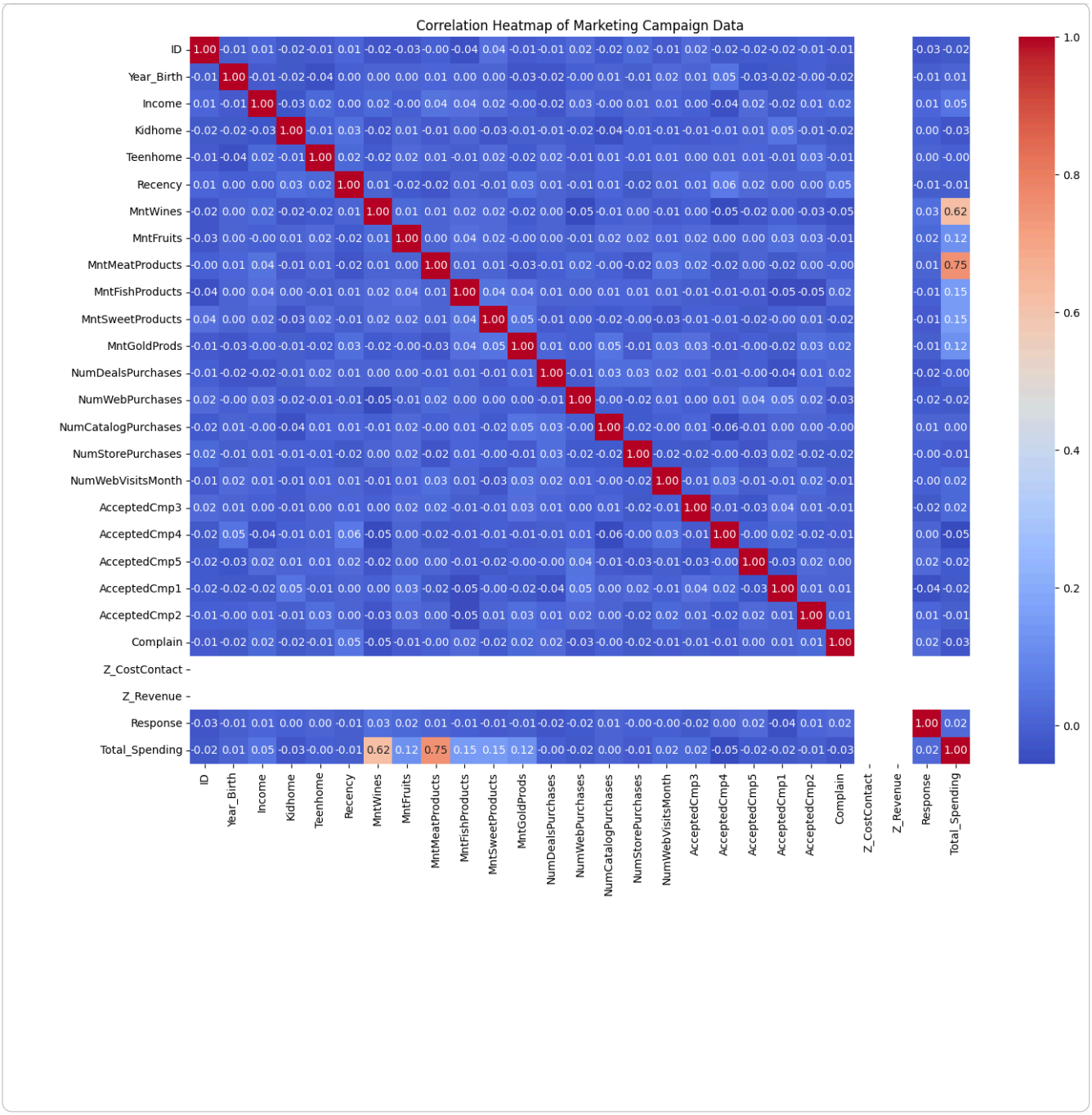
```
['ID', 'Year_Birth', 'Education', 'Marital_Status', 'Income', 'Kidhome', 'Teenhome', 'Recency', 'MntWines', 'MntFruits', 'Mr
```

```
# Create Total Spending column

spending_cols = [
    "MntWines",
    "MntFruits",
    "MntMeatProducts",
    "MntFishProducts",
    "MntSweetProducts",
```

```
    "MntGoldProds"
]

df["Total_Spending"] = df[spending_cols].sum(axis=1)

print(df[["ID", "Total_Spending"]].head())
```

```
   ID  Total_Spending
0   1            2233
1   2            2280
2   3            1178
3   4            2189
4   5            2385
```

```
# Final Correlation Heatmap

numeric_df = df.select_dtypes(include="number")

plt.figure(figsize=(16, 12))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

plt.title("Correlation Heatmap of Marketing Campaign Data")
plt.show()
```




```
# Campaign performance

campaign_totals = df[campaign_cols].sum().sort_values(ascending=False)

print("Campaign Acceptance:")
print(campaign_totals)

print("\nBest Campaign:", campaign_totals.idxmax())
print("Accepted Customers:", campaign_totals.max())
```

```
Campaign Acceptance:
AcceptedCmp1    1178
AcceptedCmp4    1148
AcceptedCmp2    1138
AcceptedCmp3    1111
AcceptedCmp5    1108
dtype: int64

Best Campaign: AcceptedCmp1
Accepted Customers: 1178
```

```
# Overall campaign response rate

total_customers = len(df)
responded_customers = df["Response"].sum()

response_rate = (responded_customers / total_customers) * 100
```

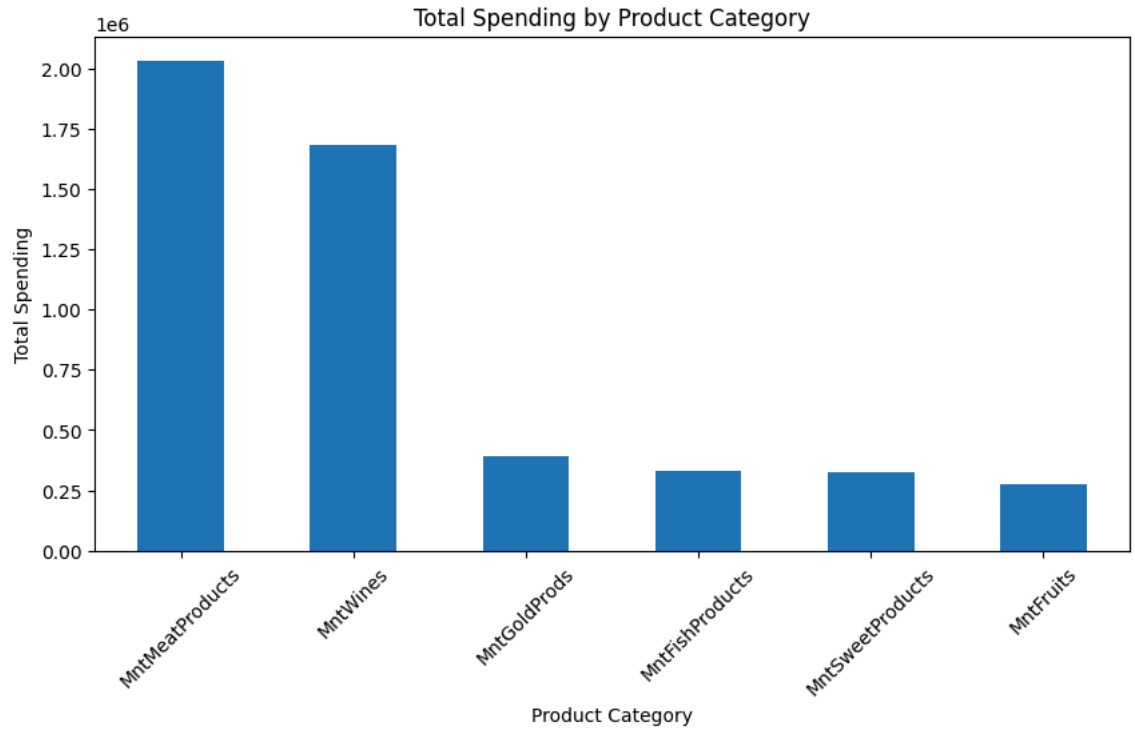
```
print("Total Customers:", total_customers)
print("Responded Customers:", responded_customers)
print("Overall Response Rate:", round(response_rate, 2), "%")
```

```
Total Customers: 2240
Responded Customers: 1108
Overall Response Rate: 49.46 %
```

```
# Total spending by campaign channel

spending_cols = [
    'MntWines',
    'MntFruits',
    'MntMeatProducts',
    'MntFishProducts',
    'MntSweetProducts',
    'MntGoldProds'
]

total_spending = df[spending_cols].sum().sort_values(ascending=False)

print("Total Spending by Product Category:")
print(total_spending)
```

```
Total Spending by Product Category:
MntMeatProducts     2031373
MntWines            1680608
MntGoldProds         389583
MntFishProducts      330359
MntSweetProducts     327428
MntFruits            277167
dtype: int64
```

```
# Bar chart for total spending by product category

total_spending.plot(kind='bar', figsize=(10,5))

plt.title("Total Spending by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Spending")
plt.xticks(rotation=45)
plt.show()
```




```
# Highest spending product category

print("Highest Spending Category:", total_spending.idxmax())
print("Highest Spending Amount:", total_spending.max())
```

```
Highest Spending Category: MntMeatProducts
Highest Spending Amount: 2031373
```

```
# Campaign acceptance vs total spending

print("Campaign Acceptance:")
print(campaign_totals)

print("\nTotal Spending:")
print(total_spending)
```

```
Campaign Acceptance:
AcceptedCmp1    1178
AcceptedCmp4    1148
AcceptedCmp2    1138
AcceptedCmp3    1111
AcceptedCmp5    1108
dtype: int64

Total Spending:
MntMeatProducts     2031373
MntWines            1680608
MntGoldProds         389583
MntFishProducts      330359
MntSweetProducts     327428
MntFruits            277167
dtype: int64
```

```
# Average income of customers

print("Average Income:", round(df["Income"].mean(), 2))
```

```
Average Income: 67622.46
```

```
# Average number of web purchases

print("Average Web Purchases:", round(df["NumWebPurchases"].mean(), 2))
```

```
Average Web Purchases: 7.03
```

```
# Compare customers based on campaign response

comparison = df.groupby("Response")[[
    "Income",
    "Total_Spending",
    "NumWebPurchases",
    "NumStorePurchases"
]].mean()

comparison.index = ["Did Not Respond", "Responded"]

comparison
```

	Income	Total_Spending	NumWebPurchases	NumStorePurchases
Did Not Respond	67222.200530	2232.887809	7.105124	7.201413
Responded	68031.398917	2264.340253	6.960289	7.201264

Next steps: Generate code with comparison | New interactive sheet


```
comparison = df.groupby("Response")[[
    "Income",
    "Total_Spending",
    "NumWebPurchases",
    "NumStorePurchases"
]].mean()

comparison.index = ["Did Not Respond", "Responded"]

comparison
```

	Income	Total_Spending	NumWebPurchases	NumStorePurchases
Did Not Respond	67222.200530	2232.887809	7.105124	7.201413
Responded	68031.398917	2264.340253	6.960289	7.201264

Next steps: Generate code with comparison | New interactive sheet


```
channel_totals = {
    "Web": df["NumWebPurchases"].sum(),
    "Catalog": df["NumCatalogPurchases"].sum(),
```

```
    "Store": df["NumStorePurchases"].sum()
}

best_channel = max(channel_totals, key=channel_totals.get)

print("Purchases by Marketing Channel:")
print(channel_totals)

print("\nBest Marketing Channel:", best_channel)
print("Total Purchases:", channel_totals[best_channel])
```

```
Purchases by Marketing Channel:
{'Web': np.int64(15755), 'Catalog': np.int64(15671), 'Store': np.int64(16131)}

Best Marketing Channel: Store
Total Purchases: 16131
```

```
print("FINAL BUSINESS INSIGHTS")
print("-----------------------")

print("1. Best Campaign:", campaign_totals.idxmax())
print("2. Highest Spending Category:", total_spending.idxmax())
print("3. Best Marketing Channel:", best_channel)
print("4. Overall Response Rate:", round(response_rate, 2), "%")
```

```
FINAL BUSINESS INSIGHTS
-----------------------
1. Best Campaign: AcceptedCmp1
2. Highest Spending Category: MntMeatProducts
3. Best Marketing Channel: Store
4. Overall Response Rate: 49.46 %
```

```
print("""
FINAL RECOMMENDATION TO THE CEO
--------------------------------

The company should focus on the most successful marketing campaign
and the marketing channel that generates the highest number of purchases.

Customers with higher spending and purchasing activity should be given
more attention through targeted marketing campaigns.

The company should also continue promoting the highest-performing
product category and improve campaigns with lower acceptance rates.

Overall, the data can help the company allocate its marketing resources
towards customers, products, campaigns, and channels that show better
performance.
""")
```

```

FINAL RECOMMENDATION TO THE CEO
--------------------------------

The company should focus on the most successful marketing campaign
and the marketing channel that generates the highest number of purchases.

Customers with higher spending and purchasing activity should be given
more attention through targeted marketing campaigns.

The company should also continue promoting the highest-performing
product category and improve campaigns with lower acceptance rates.

Overall, the data can help the company allocate its marketing resources
towards customers, products, campaigns, and channels that show better
performance.
```

```
print("BUSINESS INSIGHTS REPORT")
print("========================")
print("Marketing Campaign Analysis")
```

```
BUSINESS INSIGHTS REPORT
========================
Marketing Campaign Analysis
```

```
print("EXECUTIVE SUMMARY")
print("-----------------")

print(f"""
This analysis examines customer behavior and marketing campaign
performance using {total_customers} customer records.

The overall campaign response rate was {response_rate:.2f}%
```

```
The overall campaign response rate was {response_rate:.2f}%.
{campaign_totals.idxmax()} was the best-performing campaign based
on customer acceptance.

The highest-spending product category was {total_spending.idxmax()},
while {best_channel} was the most-used marketing channel.

These findings can help the company improve marketing campaigns,
focus on high-value customers, and allocate resources to the
best-performing products and channels.
""")
```

```
EXECUTIVE SUMMARY
-----------------

This analysis examines customer behavior and marketing campaign
performance using 2240 customer records.

The overall campaign response rate was 49.46%.
AcceptedCmp1 was the best-performing campaign based
on customer acceptance.

The highest-spending product category was MntMeatProducts,
while Store was the most-used marketing channel.

These findings can help the company improve marketing campaigns,
focus on high-value customers, and allocate resources to the
best-performing products and channels.
```

```
print("KEY FINDINGS")
print("------------")

print(f"1. The overall response rate was {response_rate:.2f}%.")

print(f"2. {campaign_totals.idxmax()} was the best-performing campaign, "
      f"with {campaign_totals.max()} accepted customers.")

print(f"3. {total_spending.idxmax()} was the highest-spending product category.")

print(f"4. {best_channel} generated the highest number of purchases.")

print("5. Customer spending and purchasing behavior can be used "
      "to identify and target valuable customers.")
```

```
KEY FINDINGS
```